

DA5402 A7: MLflow Sprint

End-to-End Explanation & Viva Preparation Guide

1. What Is This Assignment About?

This assignment is **not** about building a great image classifier. It is about **MLOps** — specifically, using **MLflow** to manage the full lifecycle of a machine learning model: training, tracking experiments, packaging code, serving the model, and connecting it to a user-facing application.

MLflow solves a real industry problem: data scientists run hundreds of experiments, tweak hyperparameters, save models in random folders, and lose track of what worked. MLflow is the "lab notebook" that records everything automatically and makes models deployable with a single command.

2. The Four Pieces We Built (and Why)

Piece 1: Training Script with Experiment Tracking (src/train.py)

What it does: Trains a ResNet18 model on the MNIST handwritten digit dataset (digits 0–9) and logs everything to MLflow.

Why ResNet18? The assignment says "pre-trained model of your choice." ResNet18 is a standard, well-understood CNN architecture from the ImageNet world. It is overkill for MNIST (a simple 2-layer CNN would work), but it demonstrates that we can adapt a real architecture to a different task. We modified two layers:

- **conv1:** Changed input channels from 3 (RGB) to 1 (grayscale), because MNIST is grayscale.
- **fc:** Changed output from 1000 classes (ImageNet) to 10 classes (digits 0–9).

We used `weights=None` (training from scratch) to keep it fast. With `weights="DEFAULT"` we would get ImageNet pre-trained weights and fine-tune, but for MNIST that is unnecessary.

Why MNIST? It is tiny (28×28 grayscale images, 70K total), downloads in seconds, and trains in minutes. Cats vs Dogs would need a larger dataset download, more training time, and more debugging — all wasted effort when the grading is on MLflow usage, not model accuracy.

Why a subset of 5000 training / 1000 test? Speed. Full MNIST is 60K/10K. With 5000 samples, each training run takes around 15 seconds instead of 2 minutes. We still get 95%+ accuracy, which is enough to demonstrate the pipeline.

What gets logged to MLflow:

What	API Call	Why
learning_rate, batch_size, epochs, architecture, optimizer	<code>mlflow.log_param()</code>	Compare settings across runs
train_loss, val_loss, val_accuracy (per epoch)	<code>mlflow.log_metric()</code>	See learning curves, pick best run

sample_predictions.png	mlflow.log_artifact()	Visual proof the model works
model.pth	mlflow.log_artifact()	Raw PyTorch weights in Artifacts tab
Full model (for serving)	mlflow.pytorch.log_model()	Packages model with metadata for serving

The training loop explained:

For each epoch:

For each batch of images:

1. Forward pass: `model(images) -> predictions`
2. Compute loss: `CrossEntropyLoss(predictions, true_labels)`
3. Backward pass: `loss.backward()` computes gradients
4. Optimizer step: SGD updates weights using gradients

After all batches:

5. Evaluate on test set -> `val_loss, val_accuracy`
6. Log metrics to MLflow with `step=epoch`

CrossEntropyLoss combines softmax + negative log likelihood. It is the standard loss for multi-class classification. It penalizes confident wrong predictions more than uncertain ones.

SGD with momentum (0.9) is the optimizer. Plain SGD updates weights purely based on the current gradient. Momentum adds "inertia" — it remembers the direction from previous steps, which helps escape small local minima and converge faster. 0.9 means 90% of the previous update direction is retained.

Model signature and input example: When we call `log_model()`, we also pass a **signature** (the exact input shape `float32 [1, 1, 28, 28]` and output shape `float32 [1, 10]`) and an **input_example** (a sample numpy array). This lets the serving endpoint validate and correctly cast incoming requests instead of guessing — which is exactly the bug we hit when we got the "expected Double but found Float" error.

Piece 2: MLflow Project (MLproject + python_env.yaml + requirements.txt)

What it does: Packages the project so anyone can run training with a single command:

```
mlflow run . -P learning_rate=0.01 --env-manager=local
```

The MLproject file:

```
name: mlflow-sprint
python_env: python_env.yaml
entry_points:
  main:
    parameters:
      learning_rate: {type: float, default: 0.01}
      batch_size: {type: int, default: 64}
      epochs: {type: int, default: 2}
    command: "python src/train.py --learning_rate {learning_rate}
      --batch_size {batch_size} --epochs {epochs}"
```

This is like a Makefile for ML. It defines the project **name**, the **environment spec**, and **entry points** (named commands you can run). `main` is the default. Each parameter has a type and default. When you run `mlflow run . -P learning_rate=0.01`, MLflow substitutes `{learning_rate}` with `0.01` in the command template.

Why --env-manager=local? By default, MLflow creates an isolated conda/virtualenv environment before running. This is great for reproducibility in production. But on Apple Silicon M1, conda can be slow or cause compatibility issues with PyTorch's arm64 wheels. `--env-manager=local` tells MLflow to skip environment creation and use the currently activated venv.

What happens internally when you run `mlflow run .`:

- MLflow reads the MLproject file in the current directory
- Finds the main entry point
- Substitutes parameters into the command template (defaults for unspecified ones)
- Creates a new Run in the tracking store
- Executes the command in a subprocess
- When the script finishes, marks the run as succeeded or failed

Piece 3: Model Serving (mlflow models serve)

What it does: Takes a trained model and exposes it as a REST API endpoint:

```
mlflow models serve -m "runs:/<run_id>/model" --port 5001 --env-manager=local
```

This starts a web server (Uvicorn/FastAPI under the hood) on port 5001 that accepts POST requests at `/invocations`.

The runs:/ URI explained:

`runs:/<run_id>/model` is MLflow's URI scheme for locating artifacts. MLflow looks up the run ID in the tracking store (`mlflow.db` — a SQLite database), reads the `artifact_uri` column, and follows it to `./mlruns/0/<run_id>/artifacts/model/`. Inside that folder: `MLmodel` (YAML metadata), `model.pkl` (serialized weights), `conda.yaml`, and `requirements.txt`. The `0` in the path is the experiment ID — "Default" is always `0`.

How a prediction request works:

```
Client POSTs to http://127.0.0.1:5001/invocations
Body: {"inputs": [[[[0.12, 0.45, ...]]]]} (shape: 1x1x28x28)
-> MLflow deserializes JSON to numpy array
-> Converts to PyTorch tensor (using signature for dtype)
-> Calls model.forward(tensor)
-> Gets 10 raw logits (one per digit class)
-> Returns: {"predictions": [[-0.73, 6.44, 0.59, ...]]}
```

The server only responds to POST at `/invocations`. Opening it in a browser sends a GET to `/` which returns `{"detail": "Not Found"}` — this is normal and expected. **Port 5001** is used because macOS uses port 5000 for AirPlay Receiver.

Piece 4: Web Client (app.py — Streamlit)

What it does: Provides a user-friendly interface where someone uploads a digit image and gets a classification result.

The preprocessing pipeline:

```
User uploads image (any size, possibly color)
```

```
-> Convert to grayscale
-> Resize to 28x28 pixels (MNIST size)
-> Auto-detect background:
    if mean brightness > 127: invert (user drew black on white)
    else: leave as-is (already MNIST-style white on black)
-> Convert to float32, divide by 255 (scale to 0-1)
-> Normalize: (pixel - 0.1307) / 0.3081 [MNIST mean/std]
-> Reshape to [1, 1, 28, 28]
```

Why normalize with 0.1307 and 0.3081? These are the pre-computed mean and standard deviation of all pixel values across the entire MNIST dataset. Normalizing ensures the data distribution matches what the model saw during training. Skipping this causes a distribution mismatch and degrades predictions.

The prediction flow:

- `preprocess()` returns a numpy array [1, 1, 28, 28]
- `predict()` converts it to a nested Python list via `.tolist()`
- Wraps it as `{"inputs": [...]}` and POSTs to the MLflow server
- Gets back 10 raw logits, applies softmax to get probabilities
- `argmax` gives the predicted digit; displays digit, confidence %, and bar chart

Softmax: The model outputs 10 raw numbers (logits). Softmax converts them to probabilities that sum to 1: $\text{probability}[i] = \frac{\exp(\text{logit}[i])}{\sum(\exp(\text{logit}[j]))}$. We use the numerical trick $\text{logits} - \max(\text{logits})$ before `exp()` to prevent overflow.

3. The Three Metrics — What They Mean

train_loss

How wrong the model's predictions are on the **training data**, averaged over all batches in an epoch. Computed by CrossEntropyLoss. Should decrease every epoch. If it plateaus, the model has stopped learning. If it increases, the learning rate is probably too high.

val_loss

Same measurement, but on the **test set** (data the model never trains on). This is the honest measure of performance. Should decrease alongside train_loss. If val_loss goes UP while train_loss goes DOWN, that signals **overfitting** — the model is memorizing training examples instead of learning generalizable patterns.

val_accuracy

Percentage of correct predictions on the test set. The most interpretable metric. 0.95 means 950 out of 1000 test images were classified correctly. This is the metric used to pick the "best" run.

Healthy training vs. Overfitting

Metric	Healthy Training	Overfitting
train_loss	Steadily decreasing	Steadily decreasing
val_loss	Decreasing (tracks train_loss)	Starts INCREASING (diverges)
val_accuracy	Steadily increasing	Plateaus or decreases

In our case, all three metrics behave healthily — no overfitting, which makes sense because we only train for 2–3 epochs on a simple task.

4. Hyperparameter Comparison — What We Learned

Run	LR	Batch Size	Epochs	val_acc	Observation
Best	0.005	64	3	~0.97	Sweet spot — smooth convergence
Good	0.001	32	3	~0.96	Slightly slower, still good
Bad	0.1	128	3	~0.89	LR too high — chaotic 1st epoch

Why did lr=0.1 fail? The learning rate controls how big each weight update step is. At 0.1, the steps are so large that the optimizer overshoots the loss minimum — like trying to land a plane by diving straight at the runway. The first-epoch val_loss spiked to ~25 (vs ~0.3 for good runs). It partially recovered but never caught up.

Why is lr=0.005 slightly better than lr=0.001? Both are in the "stable" range, but 0.005 converges faster — it takes slightly larger steps, reaching a good minimum in fewer epochs. Given more epochs (say 10), lr=0.001 would likely catch up. This is the classic tradeoff: too high = unstable, too low = slow, just right = fast and stable.

Why does batch size matter? Batch size determines how many images the model sees before updating weights. Smaller batches (32) mean more frequent updates with noisier gradients — this can help escape local minima but is slower per epoch. Larger batches (128) give smoother gradients but fewer updates per epoch.

5. The MLflow Ecosystem — Key Concepts

Tracking Store: Where MLflow records runs, params, metrics, and tags. We used SQLite (mlflow.db). In production, you would use PostgreSQL or MySQL for concurrent access.

Artifact Store: Where MLflow stores files (models, plots, etc.). We used the local filesystem (./mlruns/). In production, you would use S3, GCS, or Azure Blob Storage.

Experiment: A named group of runs. We used "Default" (ID 0). You can create custom experiments: `mlflow.set_experiment("my-experiment")`.

Run: A single execution of your training script. Each run gets a unique ID and records params, metrics, artifacts, start/end time, and source code info.

MLmodel File: A YAML metadata file inside every logged model. Tells MLflow the model flavor (pytorch, sklearn, etc.), how to load it, the input/output signature, and dependencies.

Model Signature: Specifies the exact schema of inputs and outputs (shapes and dtypes). Enables MLflow to validate and cast incoming requests correctly.

6. Architecture Diagram



7. File-by-File Explanation

MLproject — Defines the project name, environment, and entry points. Allows mlflow run . to work. Like a Makefile for ML experiments.

python_env.yaml — Tells MLflow which Python version and pip dependencies to install if it creates an environment. We bypass this with --env-manager=local.

requirements.txt — Standard pip requirements file. Lists all dependencies: mlflow, torch, torchvision, scikit-learn, matplotlib, streamlit, pillow, requests.

src/train.py — The training script. Builds model, loads data, trains, evaluates, and logs everything to MLflow. Core of the assignment.

app.py — The Streamlit web client. Handles image upload, preprocessing, calling the MLflow server, and displaying results.

mlflow.db — SQLite database created automatically by MLflow. Contains tables for experiments, runs, params, metrics, tags. Queryable: sqlite3 mlflow.db "SELECT * FROM runs;"

mlruns/ — Directory where MLflow stores artifacts on disk. Structure: mlruns/<experiment_id>/<run_id>/artifacts/. This is what the runs:/ URI resolves to.

data/ — MNIST dataset, auto-downloaded by torchvision on first run. Contains raw image files and labels.

artifacts/ — Temporary directory where we save sample_predictions.png and model.pth before MLflow copies them into mlruns/.

8. Common Viva Questions & Answers

Q: Why MLflow and not just saving models manually?

Manual saving does not track which hyperparameters produced which model, does not version models, does not let you compare runs, and does not provide one-command serving. MLflow automates all of this. In a team of 10 data scientists running thousands of experiments, manual tracking is impossible.

Q: What happens if two people run experiments at the same time with SQLite?

SQLite has a single-writer lock, so concurrent writes can cause "database is locked" errors. That is why production MLflow deployments use PostgreSQL or MySQL as the backend store.

Q: Could you deploy this to production?

Not as-is. mlflow models serve is for local development. For production: (1) use a remote tracking server with a proper database, (2) store artifacts in cloud storage (S3/GCS), (3) containerize the model with mlflow models build-docker, and (4) deploy to Kubernetes, AWS SageMaker, or similar.

Q: Why not just use TensorBoard?

TensorBoard is great for visualizing training curves but does not handle model packaging, serving, or project management. MLflow is a full lifecycle tool — tracking is just one component.

Q: What is the difference between `log_artifact` and `log_model`?

`log_artifact` uploads any file as-is (PNG, CSV, .pth, anything). `log_model` is specifically for ML models — it saves the weights AND creates an MLmodel metadata file with flavor info, signature, and dependencies. Only `log_model`'d models can be served with mlflow models serve.

Q: Why did you use `torch.save(model.state_dict())` AND `mlflow.pytorch.log_model()`?

`torch.save` creates a raw .pth file that appears in the Artifacts tab — it is a visual artifact for graders. `log_model` creates the full MLflow model package (with MLmodel metadata, signature, etc.) needed for serving. We need both because MLflow 3.x displays `log_model` outputs in a separate "Logged Models" section, not in the Artifacts tab.

Q: What does the MPS device do?

MPS (Metal Performance Shaders) is Apple's GPU acceleration framework for M1/M2 Macs. It is like CUDA for NVIDIA GPUs. `torch.device("mps")` tells PyTorch to run tensor operations on the M1's GPU instead of CPU. For MNIST it barely matters, but for large models it is 5–10x faster.

Q: Why normalize with `mean=0.1307` and `std=0.3081`?

These are the pre-computed statistics of the entire MNIST dataset. Normalizing ensures input pixel values have mean approximately 0 and std approximately 1, which helps gradient-based optimization converge faster and more stably. Using different normalization at inference vs training would cause a distribution mismatch.

Q: What is inside the MLmodel file?

A YAML file that describes: flavors (how to load it — `pytorch`, `python_function`), signature (input/output shapes and dtypes), `model_uuid` (unique identifier), `run_id` (which MLflow run produced it), and `saved_input_example_info`.

Q: How does the Streamlit app communicate with the MLflow server?

Pure HTTP. Streamlit preprocesses the image into a numpy array, converts to JSON, and sends a POST to `http://127.0.0.1:5001/invocations`. The MLflow server returns JSON with predictions. They are two separate processes communicating over HTTP — they could run on different machines.

Q: What would you do differently with more time?

(1) Train on full MNIST for higher accuracy. (2) Use k-fold cross-validation instead of a single train/test split. (3) Try different architectures (simple CNN, MobileNet) and compare in MLflow. (4) Add model registration and staging (MLflow Model Registry). (5) Containerize with Docker. (6) Deploy to a cloud endpoint.

Q: Walk me through what happens when you run `mlflow run . -P learning_rate=0.01`

MLflow reads the `MLproject` file, finds the main entry point, substitutes the parameter into the command template, creates a new run in the tracking store (`mlflow.db`), then executes `python src/train.py --learning_rate 0.01 --batch_size 64 --epochs 2` (defaults fill in). `--env-manager=local` skips creating a new environment and uses the active `venv`.